

建立知识地图：

模型怎么学会能力 → 怎么对齐人类偏好 → 怎么高效推理 → 怎么接工具和外部知识 → 怎么做成 Agent → 怎么评估效果 → 怎么控制成本和风险 → 怎么在行业场景里落地。

1. Transformer 与 Attention

这是大模型的底层架构。你至少要理解：

Transformer 为什么取代 RNN/LSTM？Attention 到底在做什么？

通俗说，Attention 就是模型在读一句话时，会动态判断：

“当前这个 token 最应该关注前面哪些 token？”

比如：

“苹果发布了新手机，它的销量很好。”

模型要判断“它”指的是“新手机”，不是“苹果公司”，这就需要 Attention。

不一定要推公式，但要能讲清楚：

- Transformer 的核心是 Attention + Feed Forward + 残差连接 + LayerNorm；
- Attention 解决的是长距离依赖和并行训练问题；
- 大模型本质上是基于上下文预测下一个 token。

大模型不是“理解世界”后再回答，而是在海量语料训练中学会了用上下文预测下一个 token；但当模型规模、数据规模和训练方式足够强时，这种预测能力会涌现出推理、归纳、翻译、代码、规划等高级能力。

2. Token、Embedding 与 Next Token Prediction

这是理解大模型训练目标的基础。

大模型不是直接读“字”或“词”，而是把文本切成 token，再转成向量 embedding。

比如：

“金融科技转型”

可能会被切成若干 token，每个 token 会被映射成一个高维向量。

大模型训练的基本目标是：

给定前面的 token，预测下一个 token。

比如：

中国的首都是 ____

模型希望预测出“北京”。

这背后对应的是：

- tokenizer：怎么切分文本；
- embedding：怎么把 token 变成向量；
- logits：模型对下一个 token 的概率分布；
- loss：预测结果和真实答案之间的差距；
- 训练：不断降低 loss。

大模型训练不是一开始就教它“做人类助手”，预训练阶段本质上是在做大规模文本压缩和语言规律学习。

3. 预训练、后训练与对齐

一个大模型大致会经历：

预训练 → 监督微调 SFT → 偏好对齐 RLHF/DPO/RLAIF → 安全对齐 → 产品化调优

它们的区别是：

阶段	目的	通俗解释
预训练	学知识和语言规律	让模型读完整个互联网、书籍、代码、论文
SFT	学会按指令回答	教模型像助手一样回答问题
RLHF / DPO	学会人类更喜欢什么答案	让模型知道什么叫“更好、更安全、更有帮助”
安全对齐	控制风险	避免违法、危险、恶意输出
产品调优	适配场景	比如金融问答、代码助手、Agent、客服等

预训练决定模型的基础能力上限，后训练决定模型是否好用、是否稳定、是否符合人类偏好。很多产品体验上的差异，不完全来自底座大小，也来自后训练数据、偏好优化和系统工

程。

4. Scaling Law 与涌现能力

Scaling Law 是大模型行业最核心的底层信仰之一。

它的意思是：

模型规模、数据规模、计算量增加时，模型性能往往会按某种规律提升。

这解释了为什么过去几年大家疯狂堆：

- 参数量；
- 训练数据；
- GPU 算力；
- 上下文长度；
- 推理算力。

但你还要理解另一个概念：**涌现能力**。

也就是模型变大到一定程度后，突然出现小模型不具备的能力，比如：

- 多步推理；
- 代码生成；
- 工具调用；
- 数学能力；
- 复杂指令遵循；
- 长文本综合。

Scaling Law 解释了“为什么大”有价值，但并不意味着单纯堆参数就能赢。现在行业竞争已经从单纯参数规模，转向数据质量、训练效率、推理成本、后训练能力、Agent 能力和场景落地效率的综合竞争。

5. 推理、CoT 与 Test-Time Compute

这是 DeepSeek 这类公司尤其重视的方向。

大模型回答问题有两类能力：

一种是“直接生成答案”，另一种是“边推理边生成答案”。

CoT，也就是 Chain of Thought，指的是让模型通过中间推理步骤来解决复杂问题。

但现在更重要的是 **Test-Time Compute**：

不是只在训练时投入算力，而是在推理阶段投入更多计算，让模型想得更久、搜索更多路径、验证更多答案，从而提高复杂任务表现。

比如数学题、代码题、复杂规划题，模型不是一次性吐答案，而是可能进行：

- 多步思考；
- 自我检查；
- 多路径采样；
- 反思修正；
- 工具验证；
- 最终汇总。

传统大模型更像“凭直觉回答”，推理模型更像“给它更多草稿纸和检查时间”。未来模型能力提升，不只来自训练更大的模型，也来自推理阶段更聪明地使用计算资源。

6. RAG：检索增强生成

RAG 是 AI 应用落地最重要的技术之一。

它解决的问题是：

大模型自己的参数知识不一定新、不一定准，也不知道企业内部资料，所以需要先检索外部知识，再让模型基于检索结果回答。

典型流程是：

用户问题 → 向量检索/关键词检索 → 找到相关文档片段 → 拼进 prompt → 模型生成答案

它特别适合金融场景，比如：

- 研究报告问答；
- 制度文件检索；
- 客户纪要分析；
- 投研知识库；
- 产品合同审核；
- 合规条款查询；
- 内部管理制度问答。

但你要知道，RAG 不只是“把文档塞给模型”。真正难的是：

- 文档怎么切 chunk；
- 向量检索和关键词检索怎么结合；
- 如何重排 rerank；
- 如何避免召回错误；
- 如何做引用溯源；
- 如何判断模型是不是胡编；
- 如何处理表格、PDF、扫描件、长文档；
- 如何评估答案质量。

RAG 的核心不是“接一个向量库”，而是把企业知识转化成可检索、可引用、可验证的上下文系统。真正决定效果的是数据清洗、切分策略、召回质量、重排、引用约束和评估闭环。

7. Agent、工具调用与 Function Calling

这是你从“AI 使用者”升级为“AI 系统构建者”的关键概念。

普通 Chatbot 是：

用户问 → 模型答

Agent 是：

用户给目标 → 模型拆任务 → 调工具 → 观察结果 → 再决策 → 最终完成任务

比如用户说：

帮我分析最近一周港股市场异动，并生成一页汇报材料。

Agent 可能会：

1. 查询市场数据；
2. 计算涨跌幅；
3. 判断异常指标；
4. 搜索相关新闻；
5. 归因；
6. 生成结构化观点；
7. 输出 Word/PPT。

这里的核心是 **工具调用 Function Calling** 。

模型本身不会真的查数据库、发邮件、跑 Python、调用 Wind、生成 Excel，它需要通过工具接口完成。

你要理解：

- tool schema 是什么；
- ReAct 是什么；
- planning / acting / observing 是什么；
- 工具调用失败如何降级；
- 多 Agent 不一定比单 Agent 好；
- Agent 的难点是稳定性、状态管理、边界控制和结果评估。

Agent 的本质不是让模型“更聪明地聊天”，而是让模型成为一个可以调用工具、分解任务、读取环境反馈并持续推进目标的执行系统。它的难点不在 demo，而在稳定性、可控性、权限、安全和评估。

8. Fine-tuning、LoRA 与领域适配

你是金融背景，这个概念非常重要。

很多人会误以为：

企业要做金融大模型，就必须重新训练一个大模型。

其实不一定。

常见路线有三种：

路线	适合场景
Prompt Engineering	轻量任务、规则明确
RAG	需要接入企业知识、保持知识更新
Fine-tuning / LoRA	需要模型学习特定风格、格式、任务模式
从头训练大模型	极少数有海量数据和算力的大厂适合

LoRA 是一种低成本微调方法，可以只训练少量额外参数，而不是全量更新整个大模型。

如果问题是“模型不知道某些知识”，优先考虑 RAG；如果问题是“模型知道但不会按特定方式做”，可以考虑微调；如果问题是“模型基础能力不足”，才需要更强底座。

9. Evaluation：大模型评估体系

大模型不是“看起来回答不错”就算好。你要能评估它。

评估包括：

- 基础能力评估：数学、代码、推理、知识、语言；
- 指令遵循评估：是否按要求输出；
- 事实性评估：有没有幻觉；
- RAG 评估：召回是否准确、引用是否正确；
- Agent 评估：任务完成率、工具调用成功率；
- 安全评估：是否输出违规内容；
- 产品评估：用户满意度、留存、转化、成本；
- 行业评估：金融场景下是否符合口径、制度、合规边界。

没有评估，就没有优化。大模型应用不是写完 prompt 就结束，而是要定义任务集、标准答案、评分规则、错误分类和迭代机制。

10. AI 系统工程：成本、延迟、上下文、安全与产品化

最后一个概念不是单点技术，而是系统工程。

真正的大模型产品，难点不只是“模型回答得好”，而是：

- 成本能不能承受；
- 延迟能不能接受；
- 并发能不能扛住；
- 上下文会不会爆；
- 输出能不能稳定；
- 数据会不会泄露；
- 权限边界是否清楚；
- 失败时有没有 fallback；
- 日志能不能追踪；
- 用户体验是否顺畅；
- 是否能持续迭代。

比如你做一个金融研报 Agent，不能只问：

模型能不能写一段分析？

而要问：

数据从哪里来？
口径是否一致？
输出是否可追溯？
幻觉怎么控制？
调用失败怎么办？
成本是多少？
能不能批量跑？
能不能生成 Word/PPT？
高管能不能直接使用？

这就是从“会用 AI”到“能做 AI 系统”的分水岭。

大模型公司真正需要的人，不只是会调 prompt 的人，而是能把模型能力封装成稳定、可评估、可交付、可规模化应用系统的人。
